

Step-6: Open MySQL in Docker

```
docker exec -it ecommerce-db mysql -uroot -proot
```

Step-7: Insert Data into Products Table

```
show databases;           // show databases
use ecommerce;            // Switch to ecommerce database
show tables;              // Show tables in ecommerce database
select * from products;   // Show records in product table
```

```
// Now insert data into Products table
```

```
INSERT INTO products (name, category, price, image_path) VALUES
('Gaming PC', 'computers', 1200.00, 'gaming_pc.jpeg'),
('Office Desktop', 'computers', 800.00, 'office_desktop.jpeg'),
('Mini PC', 'computers', 500.00, 'mini_pc.jpeg'),
('Workstation', 'computers', 2500.00, 'workstation.jpeg');
```

Step-8: Now check whether data is inserted or not

```
select * from products      //To check in mysql
                             (or)
localhost:8081/api/products //
```

Step-9: Now delete docker images which were build earlier

```
docker compose down
```

Step-10: Now again build the images and check whether the data is existing or not

```
docker compose up -d --build
```

```
docker exec -it ecommerce-db mysql -uroot -proot
```

Note: Now check whether the data is existing or not following Step-7 & Step-8

Note: Data which you stored in DATABASE will be deleted upon deleting images. To overcome this, i need to use volume to store data permanently even though the images were deleted.

Database with Voume

Step-1: Database with Volume

You MUST include new service “volume” in docker-compose.yml file and make sure the YML file is defined based on Step-3

If you use dockerfile

services:

db:

image: mysql:8.0

container_name: ecommerce-db3

environment:

MYSQL_ROOT_PASSWORD: root

MYSQL_DATABASE: ecommerce

volumes:

- mysql_data:/var/lib/mysql # ← persistent MySQL data

healthcheck:

test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "-uroot", "-proot"]

interval: 5s

timeout: 3s

retries: 20

backend:

build: .

container_name: ecommerce-backend3

```
depends_on:
  db:
    condition: service_healthy
environment:
  SERVER_PORT: 8081
  SPRING_DATASOURCE_URL:
jdbc:mysql://db:3306/ecommerce?useSSL=false&allowPublicKeyRetrieval=true&serverTimezon
e=UTC
  SPRING_DATASOURCE_USERNAME: root
  SPRING_DATASOURCE_PASSWORD: root
ports:
  - "8081:8081"
restart: unless-stopped

volumes:
  mysql_data: {}
```

(or)

If you use dockerfile.one

```
services:
  db:
    image: mysql:8.0
    container_name: ecommerce-db3
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: ecommerce
    volumes:
      - mysql_data:/var/lib/mysql # ← persistent MySQL data
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "-uroot", "-proot"]
      interval: 5s
      timeout: 3s
      retries: 20

  backend:
    build:
```

```
    context: .
    dockerfile: dockerfile.one
  container_name: ecommerce-backend3
  depends_on:
    db:
      condition: service_healthy
  environment:
    SERVER_PORT: 8081
    SPRING_DATASOURCE_URL:
jdbc:mysql://db:3306/ecommerce?useSSL=false&allowPublicKeyRetrieval=true&serverTimezon
e=UTC
    SPRING_DATASOURCE_USERNAME: root
    SPRING_DATASOURCE_PASSWORD: root
  ports:
    - "8081:8081"
  restart: unless-stopped

volumes:
  mysql_data: {}
```